Cost of NIST Post-Quantum Cryptography on Constrained Wireless Devices: Measurements of ML-KEM and ML-DSA Against Classical Baselines

Iqra Batool, Ph.D.
University of New Brunswick, Fredericton, Canada
Technical report, measurements taken 2026-10-11

Abstract. Federal policy now requires a transition to the NIST post-quantum standards ML-KEM (FIPS 203) and ML-DSA (FIPS 204). This report measures their computation time and message sizes at all standardized parameter sets on Google Colab cloud VM, against classical X25519, ECDSA P-256, and Ed25519. On Google Colab cloud VM, an ML-KEM-768 encapsulation took 28 microseconds and an ML-DSA-65 signature took 172 microseconds. Message size is the larger burden: one ML-KEM-768 exchange sends 2,272 bytes versus 64 for X25519, and an ML-DSA-65 public key plus signature is 5,261 bytes versus 96 for Ed25519. Code and raw data are available at [code and data repository URL].

1. Motivation

Executive Order 14412 (June 22, 2026) directs federal high-value systems to adopt post-quantum key establishment by December 31, 2030 and post-quantum signatures by December 31, 2031, and directs assistance to critical-infrastructure operators making the same transition. Constrained wireless devices, such as sensors, radios, and gateways with limited energy, memory, and airtime, are the hardest part of that migration. This report provides reproducible measurements as a first step toward an open benchmark and integration guide for such devices.

2. Setup

Device	Board / CPU	Arch	Cores	RAM (MB)	liboqs	Python	Runs per op
Google Colab cloud VM	x86_64	x86_64	2	12976	0.16.0	3.13.16	1000

Each operation was run the stated number of times after 20 warm-up runs; medians and 95th percentiles are reported. ML-KEM and ML-DSA use liboqs through liboqs-python; classical algorithms use the Python cryptography library (OpenSSL). Signatures cover a 256-byte message, representative of a control message or firmware-manifest digest. Timing includes Python call overhead, which affects classical and post-quantum operations alike.

3. Results

Figure 1. Median time per operation on each device (log scale). Blue: post-quantum. Orange: classical.

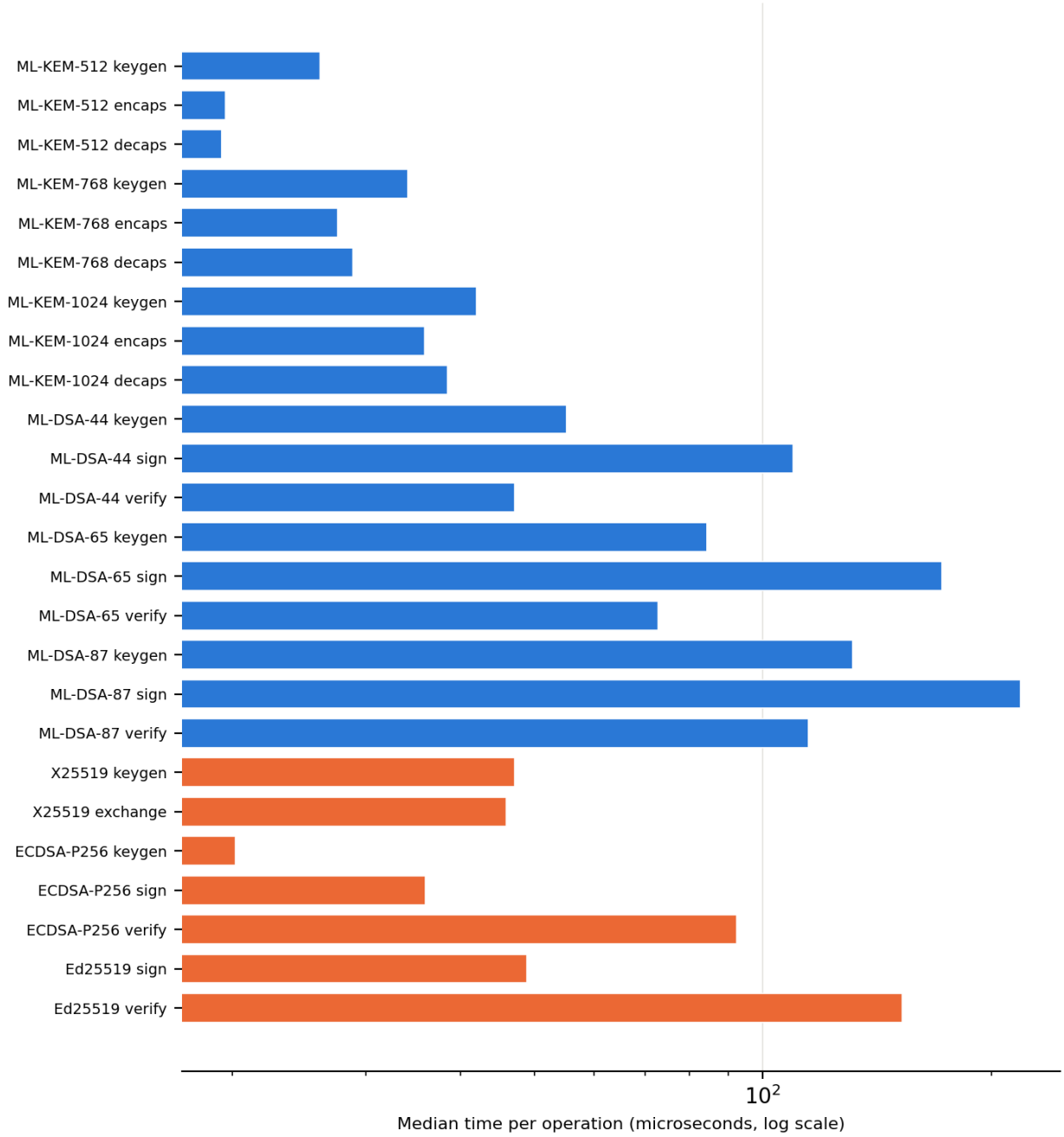
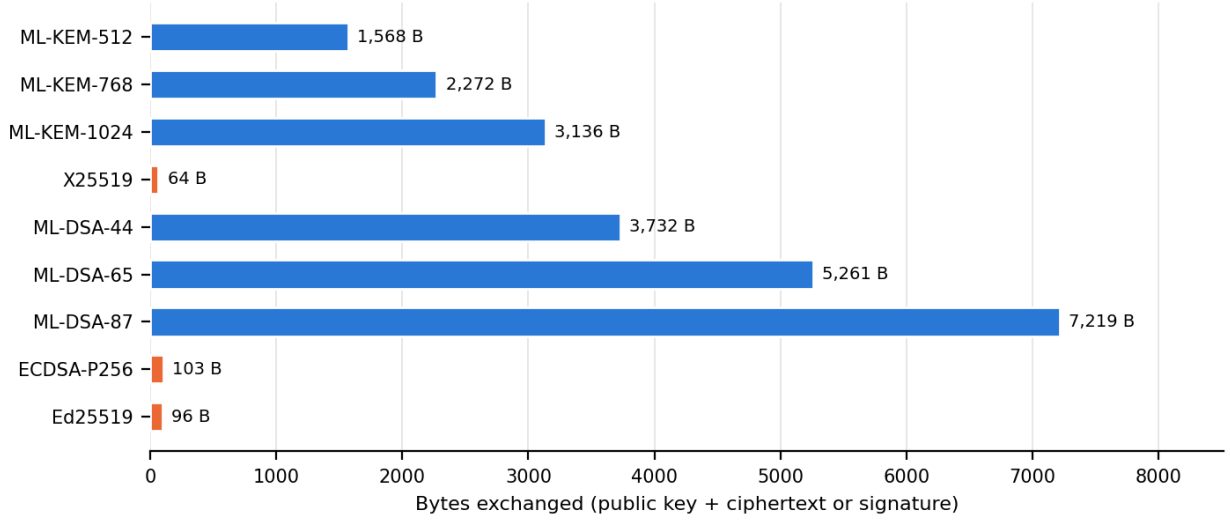

ML-KEM-512 keygen
ML-KEM-512 encaps
ML-KEM-512 decaps
ML-KEM-768 keygen
ML-KEM-768 encaps
ML-KEM-768 decaps
ML-KEM-1024 keygen
ML-KEM-1024 encaps
ML-KEM-1024 decaps
ML-DSA-44 keygen
ML-DSA-44 sign
ML-DSA-44 verify
ML-DSA-65 keygen
ML-DSA-65 sign
ML-DSA-65 verify
ML-DSA-87 keygen
ML-DSA-87 sign
ML-DSA-87 verify
X25519 keygen
X25519 exchange
ECDSA-P256 keygen
ECDSA-P256 sign
ECDSA-P256 verify
Ed25519 sign
Ed25519 verify
10^2
Median time per operation (microseconds, log scale)

Table: timing on Google Colab cloud VM

Algorithm	Operation	Median (us)	p95 (us)
ML-KEM-512	keygen	26.1	33.5
ML-KEM-512	encaps	19.6	26.3
ML-KEM-512	decaps	19.4	23.8
ML-KEM-768	keygen	34.1	42.2
ML-KEM-768	encaps	27.6	33.5
ML-KEM-768	decaps	28.8	58.5
ML-KEM-1024	keygen	42.0	52.8
ML-KEM-1024	encaps	35.8	48.0
ML-KEM-1024	decaps	38.5	45.5
ML-DSA-44	keygen	55.2	99.8
ML-DSA-44	sign	109.6	294.4
ML-DSA-44	verify	47.1	62.2
ML-DSA-65	keygen	84.5	103.5
ML-DSA-65	sign	172.2	462.8
ML-DSA-65	verify	72.9	84.5
ML-DSA-87	keygen	131.5	155.4
ML-DSA-87	sign	218.4	511.1
ML-DSA-87	verify	114.8	189.2
X25519	keygen	47.1	54.5
X25519	exchange	45.9	52.0
ECDSA-P256	keygen	20.2	32.0
ECDSA-P256	sign	36.0	44.0
ECDSA-P256	verify	92.3	134.0
Ed25519	sign	48.9	78.3
Ed25519	verify	152.7	167.7

Figure 2. Bytes exchanged per key establishment or per signed message.



4. What this means for wireless devices

Exchange	Bytes	802.15.4 frames (~100 B each)	LoRaWAN packets (51 B max)
X25519	64	1	2
ML-KEM-512	1,568	16	31
ML-KEM-768	2,272	23	45
ML-KEM-1024	3,136	32	62
Ed25519	96	1	2
ECDSA-P256	103	2	3
ML-DSA-44	3,732	38	74
ML-DSA-65	5,261	53	104
ML-DSA-87	7,219	73	142

Frame counts are illustrative: they divide the bytes by a typical usable payload and ignore fragmentation headers, retransmissions, and certificate chains, all of which add more. Even so, they show the central problem: on low-power links, post-quantum protection multiplies the number of transmissions, and each transmission costs energy and airtime. On a fast processor computation is minor; on constrained links the bytes dominate. This is why device-specific measurement, gateway-side handling, and security-aware scheduling are needed.

5. Limitations

These are single-device microbenchmarks. They do not include radio transmission, energy measured at the device, end-to-end protocol handshakes, or microcontroller-class hardware, and Python call overhead inflates very short operations. Results describe algorithm cost, not FIPS 140 module validation.

6. Next steps

Next steps are measurements on microcontroller-class boards (for example with the pqm4 framework), energy measurement per operation, hybrid key exchange in a full protocol handshake, signed firmware updates over constrained links, and use of the measured costs in security-aware resource allocation.

Data availability

Benchmark script, raw CSV results, and device descriptions: [code and data repository URL]